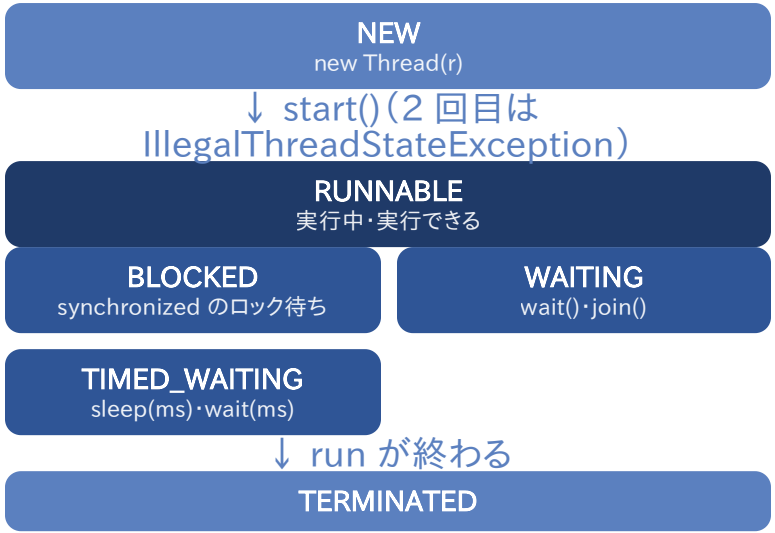
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 5 並列処理　#12

スレッドと同期

Thread・Runnable・状態・割り込み、synchronized・volatile、wait／notify・ReentrantLock・AtomicInteger

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の Thread・Object・ReentrantLock・AtomicInteger、JLS 8.4.3.6・14.19・17.4、The Java Tutorials の Concurrency）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 スレッドの一生




```
Thread t = new Thread(() -> {
  try { Thread.sleep(200); } catch (InterruptedException e) {}
});
t.getState()   // NEW
t.start(); Thread.sleep(50);
t.getState()   // TIMED_WAITING
t.join();
t.getState()   // TERMINATED
```

> RUNNING という状態はない。

出典：API Thread.State・getState・start／JDK 17 での実行結果

第1章

スレッドの作成と状態

start と run、join・sleep・interrupt。

作り方と start／run

```
class T extends Thread {
  public void run() { print("run:" + currentThread().getName()); }
}
T t = new T(); t.setName("w");
t.run();     // run:main  ← ふつうの呼び出し
t.start();   // run:w     ← 新しいスレッド

new Thread(() -> Thread.sleep(10));
// × unreported exception InterruptedException
new Thread(() -> "a");
// × lambda body is not compatible with a void functional interface
```

項目	初期値など
名前	Thread-0, Thread-1 …（new Thread(r, "名前") で指定）
優先度	5（1〜10）
デーモン	作ったスレッドと同じ（main からなら false）
setDaemon	start の前に呼ぶ
スレッドの中の例外	そのスレッドだけ終わる（main は続く）

出典：API Thread・Runnable／JDK 17 でのコンパイルと実行の結果

割り込み

```
Thread t = new Thread(() -> {
  try { Thread.sleep(5000); println("woke"); }
  catch (InterruptedException e) {
    println("interrupted " + currentThread().isInterrupted());
  }
});
t.start(); t.interrupt();
// interrupted false（例外と同時に状態はクリア）

Thread.currentThread().interrupt();
Thread.sleep(1000);   // すぐ InterruptedException
```

メソッド	中身
t.interrupt()	割り込みの状態にする
t.isInterrupted()	状態を返す（クリアしない）
Thread.interrupted()	今のスレッドの状態を返してクリア（2 回で true, false）

> sleep・wait・join は InterruptedException（チェック例外）を投げる。

出典：API Thread.interrupt・isInterrupted・interrupted・sleep／JDK 17 での実行結果

第2章

synchronized と排他制御

どのロックを使うか。

競合状態と synchronized

```
static int count = 0;
static void inc() { count++; }   // 読む→足す→書く
// 4 スレッド × 10 万回 → 400000 にならないことがある

static synchronized void inc() { count++; }
// 必ず 400000

AtomicInteger c; c.incrementAndGet();
// ロックなしで必ず 400000
```

書き方	ロック
synchronized void m1()	this
static synchronized void m2()	Main.class
synchronized (obj) { }	obj

synchronized (m) で m を持っている間：m.m1() は待つ、m2() は待たない（別のロック）。

ロック待ちは BLOCKED。再入可能。例外で抜けてもロックは外れる。

出典：JLS 8.4.3.6・14.19、The Java Tutorials「Thread Interference」「Intrinsic Locks and Synchronization」／JDK 17 での実行結果

volatile と、うまく進まない状態

```
static volatile boolean running = true;
new Thread(() -> { while (running) n++; println("stopped"); }).start();
running = false;   // すぐ見える → stopped
```

	可視性	原子性
volatile	○	×（count++ は不可）
synchronized	○	○
AtomicInteger	○	○（ロックなし）

状態	中身
デッドロック	互いに相手のロックを待って永久に止まる（BLOCKED のまま）。ロックの順をそろえて防ぐ
ライブロック	動いているが譲り合い続けて進まない
スタベーション	資源をいつまでも得られない
競合状態	タイミングで結果が変わる

出典：JLS 8.3.1.4・17.4、The Java Tutorials「Atomic Access」「Liveness」／JDK 17 での実行結果

第3章

待機・通知とロック

wait／notify・ReentrantLock・AtomicInteger。

wait と notify

```
synchronized (lock) {            // 待つ側
  while (box.isEmpty()) lock.wait();   // ロックを手放す
  println("got " + box.get(0));
}
synchronized (lock) {            // 通知する側
  box.add("item");
  lock.notifyAll();
}                                // ここでロックを手放す
// got item
lock.wait();   // synchronized の外 → IllegalMonitorStateException
```

	wait	sleep
クラス	Object（インスタンス）	Thread（static）
ロック	手放す	持ったまま
呼べる場所	synchronized の中だけ	どこでも
状態	WAITING（時間なし）	TIMED_WAITING

> notify は 1 つ、notifyAll は全部を起こす。while で条件を確かめ直す。

出典：API Object.wait・notify・notifyAll、The Java Tutorials「Guarded Blocks」／JDK 17 での実行結果

ReentrantLock と AtomicInteger

```
ReentrantLock l = new ReentrantLock();
l.unlock();          // IllegalMonitorStateException
l.lock(); l.lock();
l.getHoldCount()     // 2
l.tryLock()          // true（同じスレッド）→ 3
// 3 回 unlock して外れる

l.lock();
try { /* 処理 */ } finally { l.unlock(); }

// 別スレッドが持っているとき
l.tryLock()                          // すぐ false
l.tryLock(100, TimeUnit.MILLISECONDS) // 100ms 後 false
```

```
AtomicInteger c = new AtomicInteger(400000);
c.getAndIncrement()      // 400000（→400001）
c.incrementAndGet()      // 400002
c.addAndGet(5)           // 400007
c.compareAndSet(400007, 0) // true → 0
```

ReentrantReadWriteLock：読み込みロックは共有できる。読み込み中は書き込みロックを取れない（read true write false）。

出典：API ReentrantLock・ReentrantReadWriteLock・AtomicInteger／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
start と run	run の直接呼び出しは main で動く、start 2 回は IllegalThreadStateException
状態	BLOCKED（ロック待ち）・WAITING（wait・join）・TIMED_WAITING（sleep）
例外	InterruptedException はチェック例外、ロックなしの wait・notify・unlock は IllegalMonitorStateException
ロックの対象	インスタンスメソッドは this、static は Class
結果が決まるか	count++ は競合、synchronized・AtomicInteger なら決まる、volatile は可視性だけ
ReentrantLock	finally で unlock、tryLock の戻り値、保持数

出典：試験内容チェックリスト 5／本資料の問題演習（全 30 問、JDK 17 で確認）